Sprawozdanie z projektu: Problem plecakowy 0/1 Implementacja i analiza algorytmu genetycznego

Patryk Piwnicki (272572),
Michał Wróblewski (272488)

16 kwietnia 2026

1 Cel i zakres pracy

Celem projektu była implementacja algorytmu genetycznego (GA) rozwiązującego problem plecakowy 0/1 oraz zbadanie, jak cztery kluczowe parametry algorytmu wpływają na jakość uzyskiwanych rozwiązań. Zakres pracy objął przygotowanie modelu problemu, funkcji fitness, operatorów genetycznych oraz przeprowadzenie testów statystycznych (30 powtórzeń na wariant).

2 Opis problemu

Problem plecakowy 0/1 polega na wyborze przedmiotów w taki sposób, aby zmaksymalizować ich łączną wartość, nie przekraczając jednocześnie dopuszczalnej wagi plecaka. Matematycznie problem opisuje wzór:

$$\max \sum_{i=1}^{n} v_i x_i \quad \text{przy ograniczeniu} \quad \sum_{i=1}^{n} w_i x_i \leq W$$

gdzie $x_i \in \{0, 1\}$ oznacza wybór przedmiotu.

W projekcie wykorzystano instancję o następujących parametrach:

- Liczba przedmiotów: 18.
- Pojemność plecaka: 45.
- Wartość optymalna (brute-force): 84.

3 Algorytm genetyczny

Algorytm działa w pętli generacji, wykorzystując elitaryzm (2 najlepsze osobniki przechodzą dalej). Zastosowano następujące operatory:

- **Selekcja turniejowa:** losowanie 3 osobników i wybór najlepszego.
- **Krzyżowanie jednopunktowe:** wymiana fragmentów genotypu między rodzicami.

- **Mutacja bitowa:** losowa zmiana bitów z określonym prawdopodobieństwem.

Funkcja przystosowania (fitness) uwzględnia karę za przekroczenie pojemności plecaka, co pozwala eliminować niedopuszczalne rozwiązania.

Listing 1: Fragment implementacji algorytmu genetycznego (ga.py)

```
def genetic_algorithm(
    inst: KnapsackInstance,
    *,
    pop_size: int = POPULATION_SIZE,
    generations: int = GENERATIONS,
    p_cross: float = P_CROSS,
    p_mut: float = P_MUT,
    stagnation_limit: int | None = STAGNATION_LIMIT,
    seed: int | None = RNG_SEED,
    best_fitness_history: list[float] | None = None,
) -> tuple[list[int], float]:
    fit = inst.fitness
    if seed is not None:
        random.seed(seed)

    n = len(inst.weights)
    population = [random_individual(n) for _ in range(pop_size)]
    best = max(population, key=fit)
    best_fit = fit(best)
    stagnant = 0

    for _ in range(generations):
        population.sort(key=fit, reverse=True)
        if fit(population[0]) > best_fit:
            best_fit = fit(population[0])
            best = population[0][:]
            stagnant = 0
        else:
            stagnant += 1
        if best_fitness_history is not None:
            best_fitness_history.append(best_fit)
        if stagnation_limit is not None and stagnant >=
           stagnation_limit:
            break

        new_pop: list[list[int]] = []
        new_pop.extend(population[i][:] for i in range(min(
           ELITE_COUNT, len(population))))

        while len(new_pop) < pop_size:
            p1 = tournament_select(population, TOURNAMENT_K, fit)
            p2 = tournament_select(population, TOURNAMENT_K, fit)
            attempts = 0
            while p2 is p1 and attempts < 10:
                p2 = tournament_select(population, TOURNAMENT_K,
                   fit)
```

```
                attempts += 1
            if random.random() < p_cross:
                c1, c2 = crossover_one_point(p1, p2)
            else:
                c1, c2 = p1[:], p2[:]
            mutate(c1, p_mut)
            mutate(c2, p_mut)
            new_pop.append(c1)
            if len(new_pop) < pop_size:
                new_pop.append(c2)

        population = new_pop

    population.sort(key=fit, reverse=True)
    if fit(population[0]) > best_fit:
        best = population[0][:]
        best_fit = fit(best)
    return best, best_fit
```

4 Wyniki eksperymentów

4.1 Baseline (E1)

Przy domyślnych parametrach (populacja 80, mutacja 0.02) algorytm osiągał optimum w 70% przypadków przy średniej wartości 83.70.

4.2 Rozmiar populacji (E2)

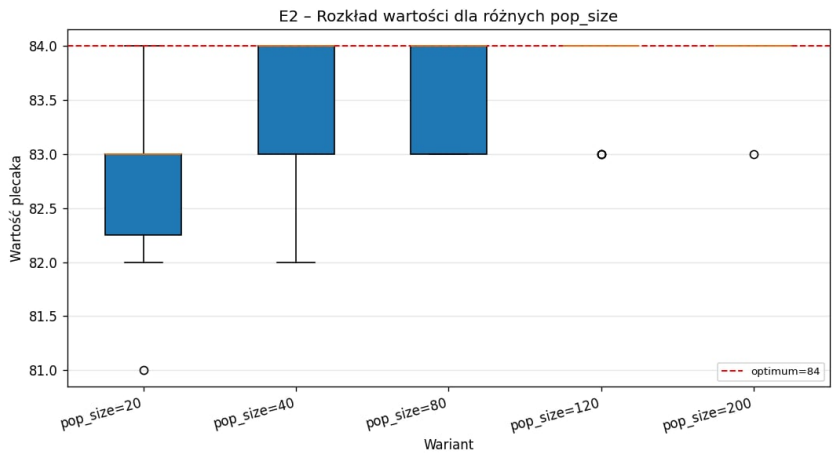
W tym eksperymencie analizowano wpływ rozmiaru populacji na jakość uzyskiwanych rozwiązań. Pozostałe parametry algorytmu zostały ustawione zgodnie z konfiguracją bazową (E1), co pozwalało na izolację wpływu wyłącznie badanego parametru.

Zwiększanie populacji wyraźnie poprawia skuteczność (Hit Rate), ale jednocześnie prowadzi do wzrostu czasu obliczeń. Większa populacja oznacza większą różnorodność rozwiązań w początkowych generacjach, co zmniejsza ryzyko przedwczesnej zbieżności do lokalnego optimum.

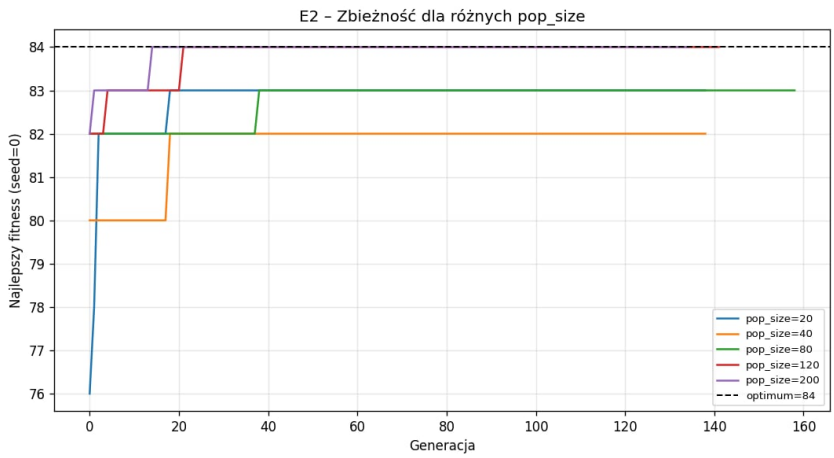
Jednocześnie należy zauważyć, że przy bardzo dużych populacjach przyrost jakości rozwiązań jest relatywnie niewielki w stosunku do zwiększonego kosztu obliczeniowego, co wskazuje na istnienie kompromisu pomiędzy jakością a czasem działania algorytmu.

pop_size	Śr. wartość	Hit%	Czas [ms]
20	82.93	23.3%	77
80	83.70	70.0%	243
200	83.97	96.7%	566

Tabela 1: Wpływ rozmiaru populacji na wyniki.



Rysunek 1: Rozkład wartości w zależności od rozmiarów populacji.



Rysunek 2: Zbieżność w zależności od rozmiarów populacji.

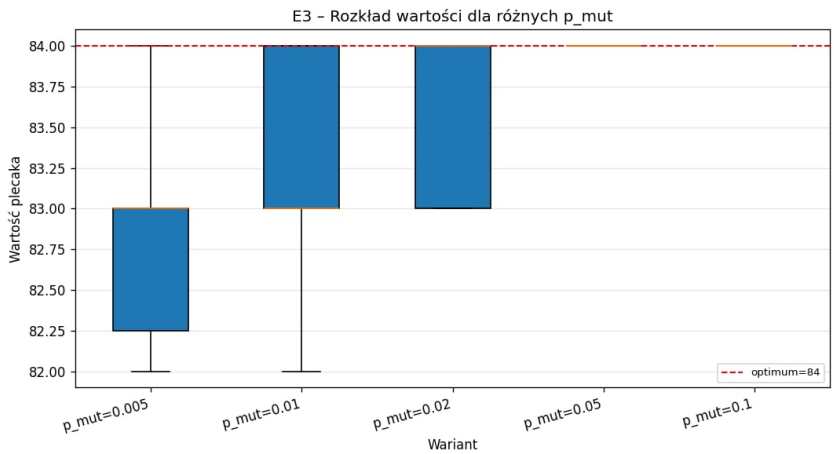
4.3 Prawdopodobieństwo mutacji (E3)

W tym eksperymencie badano wpływ prawdopodobieństwa mutacji na działanie algorytmu. Pozostałe parametry zostały ustawione zgodnie z konfiguracją bazową (E1).

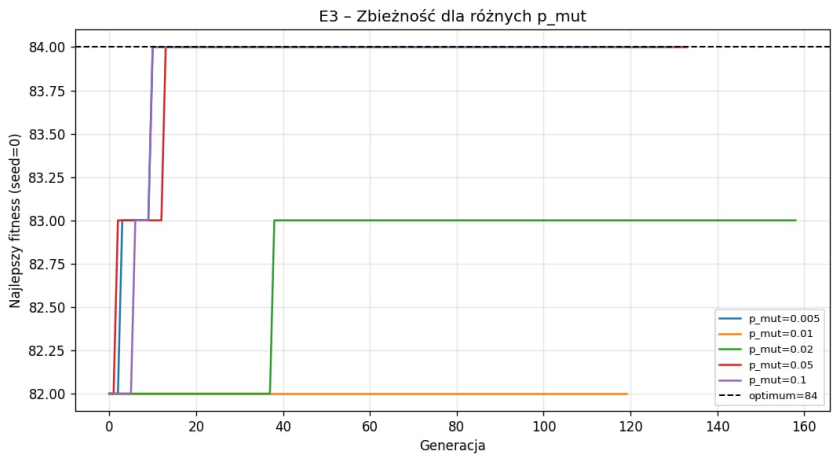
Mutacja okazała się najważniejszym parametrem wpływającym na skuteczność algorytmu. Zbyt mała wartość (np. 0.005) prowadzi do szybkiej utraty różnorodności populacji, co skutkuje utknięciem w lokalnym optimum (Hit% = 20).

Z kolei większe wartości prawdopodobieństwa mutacji zwiększają eksplorację przestrzeni rozwiązań, umożliwiając odnalezienie rozwiązania optymalnego. Dla wartości 0.05 algorytm osiągał 100% skuteczności, co wskazuje na dobre zbalansowanie eksploracji i eksploatacji.

Zbyt duże wartości mutacji mogą jednak prowadzić do nadmiernej losowości i utraty dobrych rozwiązań, dlatego konieczne jest odpowiednie dobranie tego parametru.



Rysunek 3: Rozkład wartości w zależności od mutacji.



Rysunek 4: Zbieżność wartości w zależności od mutacji.

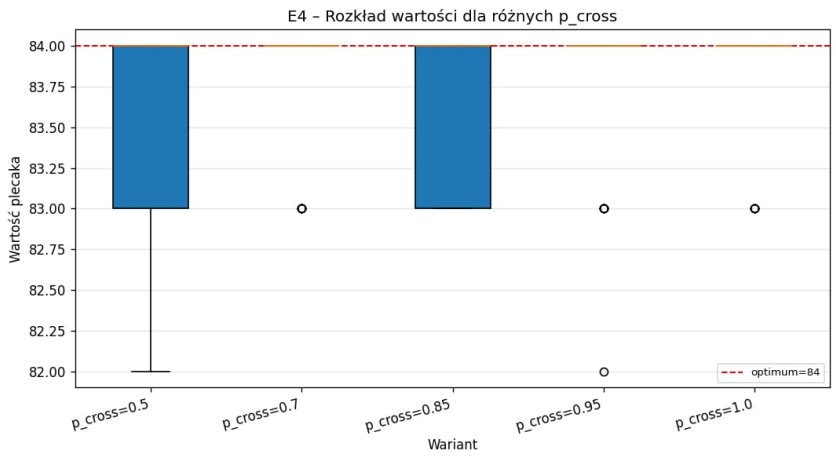
4.4 Krzyżowanie (E4) i Stagnacja (E5)

W analizie wpływu prawdopodobieństwa krzyżowania oraz limitu stagnacji pozostałe parametry zostały ustawione zgodnie z konfiguracją bazową (E1).

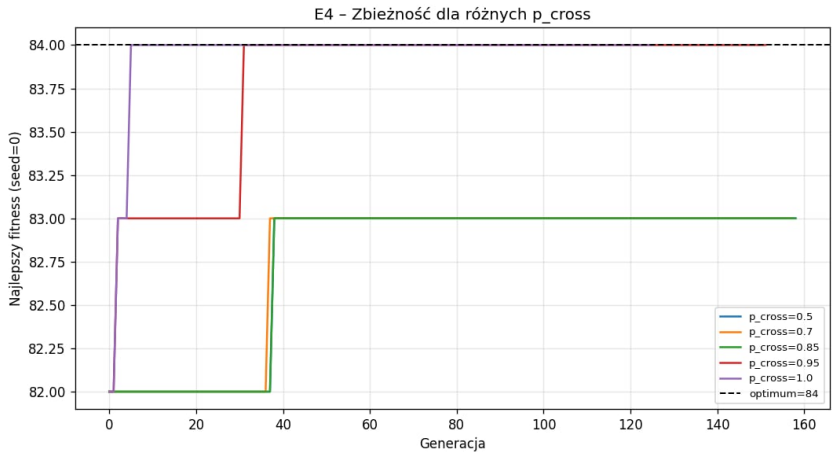
Prawdopodobieństwo krzyżowania miało stosunkowo niewielki wpływ na końcową jakość rozwiązań w badanym zakresie. Może to wynikać z faktu, że zastosowana mutacja odgrywała dominującą rolę w eksploracji przestrzeni rozwiązań.

Z kolei mechanizm wczesnego zatrzymania (stagnation limit) pozwala na skrócenie czasu działania algorytmu poprzez zakończenie obliczeń w momencie braku poprawy wyniku przez określoną liczbę generacji.

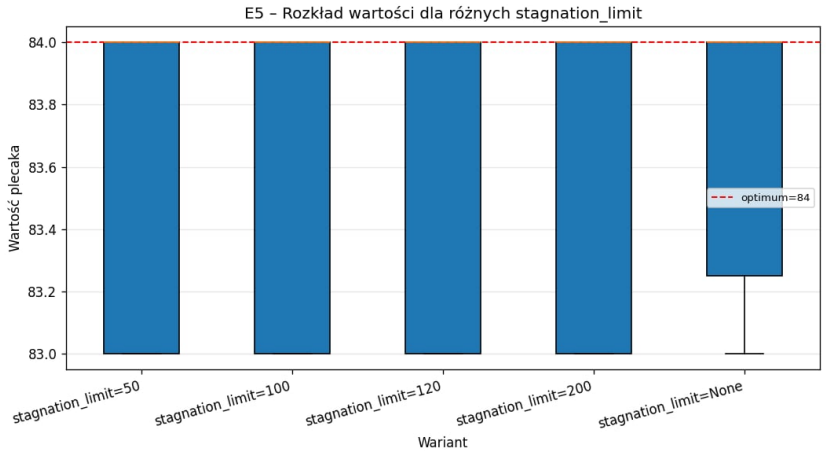
Analiza wykazała, że różne wartości parametru stagnation_limit prowadzą do bardzo zbliżonej zbieżności (co widoczne jest na wykresie), co oznacza, że algorytm osiąga stabilne rozwiązanie w podobnym czasie, niezależnie od jego ustawienia. Wartość 120 generacji stanowi rozsądny kompromis pomiędzy czasem działania a jakością wyników.



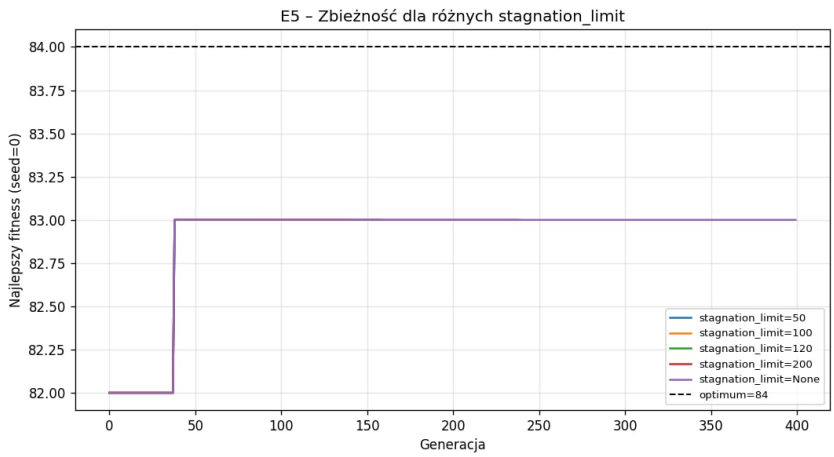
Rysunek 5: Rozkład wartości w zależności od współczynnika krzyżowania.



Rysunek 6: Zbieżność wartości w zależności od współczynnika krzyżowania.



Rysunek 7: Rozkład wartości w zależności od limitu stagnacji.



Rysunek 8: Zbieżność wartości w zależności od limitu stagnacji. Wartości dla różnych ustawień parametru stagnation_limit praktycznie się pokrywają, co wskazuje na zbliżone tempo zbieżności niezależnie od jego wartości.

5 Wnioski

Przeprowadzone eksperymenty potwierdziły, że algorytm genetyczny stanowi skuteczną metodę rozwiązywania problemu plecakowego 0/1, jednak jego efektywność w dużym stopniu zależy od odpowiedniego doboru parametrów.

Najważniejszym czynnikiem wpływającym na jakość uzyskiwanych rozwiązań okazało się prawdopodobieństwo mutacji. Zbyt niska jego wartość prowadziła do szybkiej utraty różnorodności populacji i utknięcia w lokalnych optimach, natomiast odpowiednio dobrana mutacja umożliwiała skuteczne przeszukiwanie przestrzeni rozwiązań i osiąganie optimum globalnego.

Istotny wpływ miał również rozmiar populacji. Większe populacje zwiększały stabilność wyników oraz prawdopodobieństwo znalezienia rozwiązania optymalnego, jednak odbywało się to kosztem wydłużonego czasu obliczeń. Wskazuje to na konieczność znalezienia kompromisu pomiędzy jakością rozwiązania a efektywnością czasową.

W przeciwieństwie do powyższych parametrów prawdopodobieństwo krzyżowania miało stosunkowo niewielki wpływ na końcowe wyniki, co sugeruje, że w analizowanej konfiguracji większą rolę odgrywała mutacja niż rekombinacja genów.

Mechanizm elitaryzmu oraz kara za przekroczenie pojemności plecaka okazały się kluczowe dla stabilności działania algorytmu, umożliwiając zachowanie najlepszych rozwiązań oraz eliminację rozwiązań niedopuszczalnych.

Dodatkowo zastosowanie mechanizmu wczesnego zatrzymania pozwoliło na istotne skrócenie czasu działania algorytmu bez zauważalnej utraty jakości rozwiązań, co czyni go praktycznym usprawnieniem w implementacjach rzeczywistych.